SpecialistRoute Handover — 9. Code Reference

Oct 26, 2026 · @Anas Alsirafy

The v2 code is one main notebook, one label-quality notebook and four scripts, delivered as `SpecialistRoute_Code.zip` (`notebooks/`, `scripts/`, `archive/`). Every file sets its paths in one settings block at the top; nothing else needs editing to run it.

At a glance

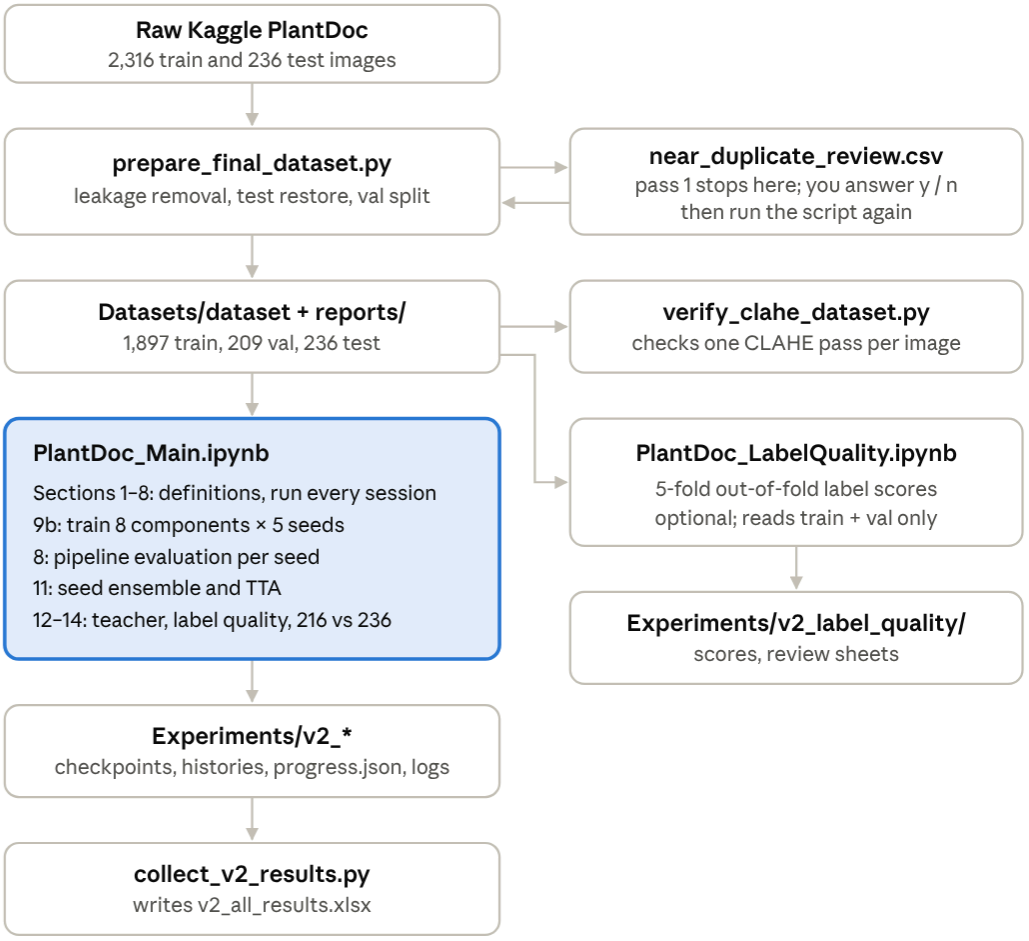
File	Folder	Purpose	GPU	When to run
`prepare_final_dataset.py` / `.ipynb`	scripts / notebooks	Builds the v2 dataset (leakage removal, test restore, validation split) and its reports	No	Once, when (re)building the data
`verify_clahe_dataset.py`	scripts	Confirms every stored image is exactly one CLAHE pass of its original	No	After building the data
`PlantDoc_Main.ipynb`	notebooks	Config, data loading, training, pipeline evaluation, ensemble, analyses	Yes	For all training and evaluation
`collect_v2_results.py`	scripts	Collects every saved result into `v2_all_results.xlsx`	No	After training or evaluation changes
`PlantDoc_LabelQuality.ipynb`	notebooks	Out-of-fold label-quality scores, review sheets, validation check	Briefly (features)	Optional analysis
`gmp_layer.py`	scripts	Generalized-mean pooling layer (v1 experiment)	—	Import before loading a GMP model
`PlantDoc_Archive_Tools.ipynb`	archive	Retired v1 cells	—	Reference only

Run everything with the `tf_env` Python (`/home/auc/tf_env/bin/python`), from a notebook kernel or a terminal.

How the files fit together

Raw data becomes results through one data script and one main notebook

Left column: the v2 workflow in order. Right column: the review step, the check and the optional analysis



v2 code flow · 4 scripts, 2 notebooks

The data script runs once (two passes). After that, all training and evaluation happen in the main notebook, and `collect_v2_results.py` turns its saved outputs into one workbook.

PlantDoc_Main.ipynb

The notebook holds the whole v2 workflow. Sections 1–8 only define things; the later sections run them.

Every session: restart the kernel, then run sections 1–8 in order. Section 1 must run first because it sets GPU memory growth.

The one cell you edit: section 2. Change paths and shared settings in `CONFIG_BASE` and per-component settings in `COMPONENTS`. Never edit `CONFIG` itself; for a one-off change use `apply_component("Tomato", epochs=10)`.

Cells that do work:

Cell	What it does	Settings at the top of the cell
9b Run everything	Trains every component for every seed (Classifier first), evaluates the pipeline after each seed, resumes after any crash	`RUN_SEEDS`, `RUN_ORDER`, `STOP_ON_ERROR`, `EVALUATE_EACH_SEED`
10 One component	Trains one component by hand (debugging, smoke tests)	`COMPONENT`, `SEEDS`
10 Evaluate seeds	Re-evaluates the pipeline for chosen seeds	`EVAL_SEEDS`
11a-2 + 11b	Caches every model's predictions (one subprocess per model), compares ensemble variants on validation	`SEEDS`; `batch_size` in `run_cache_jobs`
11c	The one test evaluation of the chosen ensemble	`CHOSEN` (`tta`, `routing`)
12b	Teacher predictions on training images and the memorisation check	`SEEDS`
13b	Label-quality scoring (same as the standalone notebook)	`LQ` in 13a
14	Accuracy on the 216 original vs 20 restored test images	`MANIFEST` path

Key functions (callable after sections 1–8, or after 11a for the section 11+ helpers):

Function	Section	Use
`apply_component(name, **overrides)`	2	Reset `CONFIG` to one component's preset, with optional overrides
`check_protocol()`	3	Pre-flight check: counts, leakage by content hash, keyword coverage, config sanity
`build_classifier_dataset_v2(include_rest=True)`	3	Build or verify the router's 7-class data folders
`get_test_dataset(model_name)`	3	Preprocessed test set; used only for final evaluation

Function	Section	Use
`train_single_split(model_name, seed_suffix)`	6	One training run with the current `CONFIG`
`run_multi_seed_experiment(model_name)`	7	Train every seed in `CONFIG["multi_seed_list"]`, reload best checkpoints, save `progress.json`
`evaluate_pipeline(seed)`	8	End-to-end test evaluation for one seed; writes logs and summary
`theoretical_pipeline(seed, df)` / `summarize_pipeline(seeds)`	8	Predicted vs measured accuracy; mean ± SD and CI over seeds
`run_all(seeds, order, ...)`	9a	The orchestrator behind 9b, with stale-run archiving
`pipeline_from_probs(split, seeds, tta, routing)`	11a	Pipeline result from cached predictions (1 seed = section 8; 5 seeds = ensemble)
`compare_variants`, `ensemble_size_curve`, `report_final`	11a	Validation comparison, accuracy vs number of seeds, final test report
`check_matches_section8(seeds)`	11a	Confirms the cache reproduces section 8 exactly
`write_cache_jobs(...)` / `run_cache_jobs(...)`	11a-2	Out-of-memory-safe prediction caching

prepare_final_dataset (.py / .ipynb)

Builds `dataset/{train,val,test}` and `reports/` from the original Kaggle data. The `.py` and `.ipynb` versions are the same code; the notebook splits it into imports, `CONFIG`, logic and a run cell.

Settings (`CONFIG` at the top):

Key	Value / meaning
`original_train`, `original_test`	Raw Kaggle PlantDoc folders
`current_train`, `current_test`	The existing cleaned, CLAHE-processed folders
`output_dir`	New output folder; `dataset/` and `reports/` are created inside
`img_size`	**224** (the stored size); 384 makes the fidelity check fail
`clahe_clip_limit`, `clahe_tile_grid`	2.0, (8, 8): must match the original CLAHE run

Key	Value / meaning
`phash_threshold`	8: Hamming distance that makes a near-duplicate candidate
`val_fraction`, `seed`	0.10, 42
`expected_test_count`	236: the run fails if the final test set differs
`include_unexplained_train`	`False`: drop current-train files with no original counterpart
`overwrite_output`	`False`: refuse to overwrite an existing output

Run: `/home/auc/tf_env/bin/python prepare_final_dataset.py`, or run the notebook's cells in order. It works in two passes:

1. **First pass** stops after finding near-duplicate candidates. It writes `reports/near_duplicate_review.csv` and a side-by-side image per pair.
2. Open the images and fill `is_duplicate` with `y` (same photo) or `n` (different photo) for every row.
3. **Second pass:** run again. It applies the exclusion rules, restores the 20 test images (with a fidelity check), makes the grouped validation split, writes all reports and checks the 236 test count.

Outputs: `dataset/{train,val,test}/<class>/`, plus in `reports/`: `manifest.csv`, `counts.csv`, `exclusion_list.csv`, `duplicate_flags.csv`, `near_duplicate_review.csv`, `moved_files.csv`.

verify_clahe_dataset.py

Re-runs the exact CLAHE pipeline on the original of every image in the final dataset and compares pixels.

- **Settings (`VERIFY`):** `final_dataset` (`/home/auc/PlantDoc/Datasets/dataset`), `original_train` / `original_test` (the **raw** Kaggle folders, never processed ones), `img_size` 224, `clahe_clip_limit` 2.0, `clahe_tile_grid` (8, 8), `max_mean_diff` 0.5.
- **Run:** `/home/auc/tf_env/bin/python verify_clahe_dataset.py`. It runs on import; there is no command-line argument.
- **Reading the result:** each image is OK (difference ≈ 0), or flagged as a large difference (CLAHE twice or never), wrong size (unprocessed), or no original found. It ends with `RESULT: ... OK` or `FIX THE FLAGGED IMAGES BEFORE TRAINING`. If everything is flagged, the original folders point at processed images.

collect_v2_results.py

Reads only files the main notebook already wrote (no TensorFlow) and writes one Excel workbook.

- **Settings (`COLLECT`):** `output_dir` (`/home/auc/PlantDoc/Experiments`), `prefix` (`v2`), `manifest` (`.../Datasets/reports/manifest.csv`), `out_file` (`.../Experiments/v2_all_results.xlsx`).
- **Run:** `/home/auc/tf_env/bin/python collect_v2_results.py` . The header comment says `/usr/bin/python3` ; use `tf_env` instead, which has pandas, scipy and openpyxl.
- **Sheets:** `README` , `component_per_seed` , `component_summary` , `pipeline_per_seed` , `pipeline_summary` , `pipeline_species` , `histories` , `pipeline_logs` (per-image predictions), `configs` , `run_state` , `warnings` .
- Skips `v2_pipeline/` and `v2_superseded/` when scanning component folders. Re-run it after any retraining; the current workbook predates the Potato fix.

PlantDoc_LabelQuality.ipynb

Scores every training image with a model that never saw it. It reads only the train and val folders and does not need the main notebook.

Cell	What it does
1 Setup	Imports; reports whether cleanlab is installed (`%pip install cleanlab` if not)
2 Configuration	`CONFIG_BASE` paths, `COMPONENTS` keywords, `RUN_COMPONENTS` order, `LQ` settings
3 Data helpers	Checks train/val classes and that every class belongs to exactly one component
4–5 Definitions	Feature-extraction worker, 5-fold scoring, contact sheets, validation check, review summary
6 Run: features	Frozen ImageNet ConvNeXt-Large features for train and val, one subprocess per split, checkpointed. Re-run until it prints `True`
7 Run: scoring	5-fold StratifiedGroupKFold + logistic regression + cleanlab per component, then the validation check (all labels vs weighted vs flagged removed)
8 Review summary	After the review CSVs are filled in, prints the share of flags confirmed per component

Main `LQ` settings: `backbone` ConvNeXtLarge, `img_size` 384, `n_folds` 5, `fold_seed` 42, `Cs` (regularisation grid), `weight_floor` 0.2, `feature_batch_size` 16, `near_dup_review_csv`, `out_dir` (`Experiments/v2_label_quality`).

Outputs: `<Component>_label_quality.csv` (score per image plus blur, brightness and contrast), `<Component>_review.csv` and `_review_sheet_p*.jpg` (flagged images, lowest score first), `proof_of_concept_val.csv`, `label_quality_summary.csv`, `review_summary.csv`. In the review CSV answer y (label wrong), n (label right, even if the image is hard) or u (can't tell).

gmp_layer.py

`GeneralizedMeanPooling2D` is a drop-in replacement for `GlobalAveragePooling2D`, from the v1 ConvGeM-NeXt experiment. It is not used by any v2 component.

- **Formula:** (mean of clip(x, eps)^ρ)^(1/ρ); ρ = 1 is average pooling, a large ρ approaches max pooling.
- **Arguments:** `init_rho=3.0`, `min_rho=1.0`, `eps=1e-6`. ρ is learnable and kept positive as softplus(raw) + `min_rho`. Inputs are clipped to ≥ eps because ConvNeXt's GELU activations can be negative.

- **Use:** `from gmp_layer import GeneralizedMeanPooling2D`, then `x = GeneralizedMeanPooling2D()(features)` in place of global average pooling.
- **Loading a saved GMP model:** the layer is registered as `custom>GeneralizedMeanPooling2D`. Import `gmp_layer` before `tf.keras.models.load_model`, otherwise loading fails.

PlantDoc_Archive_Tools.ipynb

Retired v1 cells kept for reference: data audits, Copy-Paste V2 generation (not updated for the v2 split), old evaluation cells and utilities. Its numbers and paths belong to v1. Never run it in the same kernel as `PlantDoc_Main.ipynb`; some names overlap.